

Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience

Yevgen Chebotar^{1,2} Ankur Handa¹ Viktor Makoviychuk¹
Miles Macklin^{1,3} Jan Issac¹ Nathan Ratliff¹ Dieter Fox^{1,4}

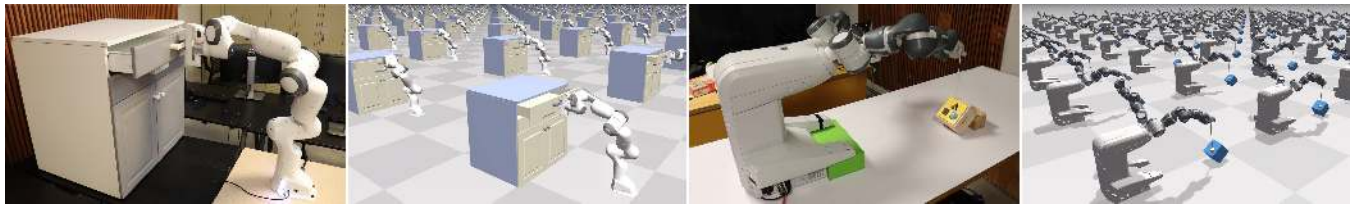


Fig. 1. Policies for opening a cabinet drawer and swing-peg-in-hole tasks trained by alternatively performing reinforcement learning with multiple agents in simulation and updating simulation parameter distribution using a few real world policy executions.

Abstract—We consider the problem of transferring policies to the real world by training on a distribution of simulated scenarios. Rather than manually tuning the randomization of simulations, we adapt the simulation parameter distribution using a few real world roll-outs interleaved with policy training. In doing so, we are able to change the distribution of simulations to improve the policy transfer by matching the policy behavior in simulation and the real world. We show that policies trained with our method are able to reliably transfer to different robots in two real world tasks: swing-peg-in-hole and opening a cabinet drawer. The video of our experiments can be found at <https://sites.google.com/view/simopt>.

I. INTRODUCTION

Learning continuous control in real world complex environments has seen a wide interest in the past few years and in particular focusing on learning policies in simulators and transferring them to the real world, as we still struggle with finding ways to acquire the necessary amount of experience and data in the real world directly. While there have been recent attempts on learning by collecting large scale data directly on real robots [1, 2, 3, 4], such an approach still remains challenging as collecting real world data is prohibitively laborious and expensive. Simulators offer several advantages, *e.g.* they can run faster than real-time and allow for acquiring large diversity of training data. However, due to the imprecise simulation models and lack of high fidelity replication of real world scenes, policies learned in simulations often cannot be directly applied on real world systems, a phenomenon also known as the *reality gap* [5]. In this work, we focus on closing the reality gap by learning policies on distributions of simulated scenarios that are optimized for a better policy transfer.

Training policies on a large diversity of simulated scenarios by randomizing relevant parameters, also known as *domain randomization*, has shown a considerable promise for the real world transfer in a range of recent works [6, 7, 8, 9].

However, design of the appropriate simulation parameter distributions remains a tedious task and often requires a substantial expert knowledge. Moreover, there are no guarantees that the applied randomization would actually lead to a sensible real world policy as the design choices made in randomizing the parameters tend to be somewhat biased by the expertise of the practitioner. In this work, we apply a data-driven approach and use real world data to adapt simulation randomization such that the behavior of the policies trained in simulation better matches their behavior in the real world. Therefore, starting with some initial distribution of the simulation parameters, we can perform learning in simulation and use real world roll-outs of learned policies to gradually change the simulation randomization such that the learned policies transfer better to the real world without requiring the exact replication of the real world scene in simulation. This approach falls into the domain of model-based reinforcement learning. However, we leverage recent developments in physics simulations to provide a strong prior of the world model in order to accelerate the learning process. Our system uses partial observations of the real world and only needs to compute rewards in simulation, therefore lifting the requirement for full state knowledge or reward instrumentation in the real world.

II. RELATED WORK

The problem of finding accurate models of the robot and the environment that can facilitate the design of robotic controllers in the real world dates back to the original works on system identification [10, 11]. In the context of reinforcement learning (RL), model-based RL explored optimizing policies using learned models [12]. In [13, 14], the data from real world policy executions is used to fit a probabilistic dynamics model, which is then used for learning an optimal policy. Although our work follows the general principle of model-based reinforcement learning, we aim at using a simulation engine as a form of parameterized model that can help us to embed prior knowledge about the world.

Overcoming the discrepancy between simulated models and the real world has been addressed through identifying simulation parameters [15], finding common feature rep-

¹NVIDIA, USA

²University of Southern California, Los Angeles, CA, USA

³University of Copenhagen, Copenhagen, Denmark

⁴University of Washington, Paul G. Allen School of Computer Science & Engineering, Seattle, WA, USA

@ychebota@usc.edu, {ahanda, vmakoviychuk, mmacklin, jissac, nratliff, dieterf}@nvidia.com

representations of real and synthetic data [16], using generative models to make synthetic images more realistic [17], fine-tuning the policies trained in simulation in the real world [18], learning inverse dynamics models [19], multi-objective optimization of task fitness and transferability to the real world [20], training on ensembles of dynamics models [21] and training on a large variety of simulated scenarios [6]. Domain randomization of textures was used in [7] to learn to fly a real quadcopter by training an image based policy entirely in simulation. Peng *et al.* [22] use randomization of physical parameters of the scene to learn a policy in simulation and transfer it to a real robot for pushing a puck to a target position. In [9], randomization of physical properties and object appearance is used to train a dexterous robotic hand to perform in-hand manipulation. Yu *et al.* [23] propose to not only train a policy on a distribution of simulated parameters, but also learn a component that predicts the system parameters from the current states and actions, and use the prediction as an additional input to the policy. In [24], an upper confidence bound on the estimated simulation optimization bias is used as a stopping criterion for a robust training with domain randomization. In [25], an auxiliary reward is used to encourage policies trained in source and target environments to visit the same states.

Combination of system identification and dynamics randomization has been used in the past to learn locomotion for a real quadruped [26], non-prehensile object manipulation [27] and in-hand object pivoting [28]. In our work, we recognize domain randomization and system identification as powerful tools for training general policies in simulation. However, we address the problem of automatically learning simulation parameter distributions that improve policy transfer, as it remains challenging to do it manually. Furthermore, as also noticed in [29], simulators have an advantage of providing a full state of the system compared to partial observations of the real world, which is also used in our work for designing better reward functions.

The closest to our approach are the methods from [30, 31, 32, 33, 34] that propose to iteratively learn simulation parameters and train policies. In [30], an iterative system identification framework is used to optimize trajectories of a bipedal robot in simulation and calibrate the simulation parameters by minimizing the discrepancy between the real world and simulated execution of the trajectories. Although we also use the real world data to compute the discrepancy of the simulated executions, we are able to use partial observations of the real world instead of the full states and we concentrate on learning general policies by finding simulation parameter distribution that leads to a better transfer without the need for exact replication of the real world environment. [31] suggests to optimize the simulation parameters such that the value function is well approximated in simulation without replicating the real world dynamics. We also recognize that exact replication of the real world dynamics might not be feasible, however a suitable randomization of the simulated scenarios can still lead to a successful policy transfer. In addition, our approach does not require estimating the reward in the real world, which might be challenging if some of the reward components can not be observed. [32] and [33] consider grounding the

simulator using real world data. However, [32] requires a human in the loop to select the best simulation parameters, and [33] needs to fit additional models for the real robot forward dynamics and simulator inverse dynamics. Finally, our work is closest to the adaptive EPOpt framework of Rajeswaran *et al.* [34], which optimizes a policy over an ensemble of models and adapts the model distribution using data from the target domain. EPOpt optimizes a risk-sensitive objective to obtain robust policies, whereas we optimize the average performance which is a risk-neutral objective. Additionally, EPOpt updates the model distribution by employing Bayesian inference with a particle filter, whereas we update the model distribution using an iterative KL-divergence constrained procedure. More importantly, they focus on simulated environments while in our work, we develop an approach that is shown to work in the real world and apply it to two real robot tasks.

III. CLOSING THE SIM-TO-REAL LOOP

A. Simulation randomization

Let $\mathcal{M} = (S, A, P, R, p_0, \gamma, T)$ be a finite-horizon Markov Decision Process (MDP), where S and A are state and action spaces, $P : S \times A \times S \rightarrow \mathbb{R}_+$ is a state-transition probability function or probabilistic system dynamics, $R : S \times A \rightarrow \mathbb{R}$ a reward function, $p_0 : S \rightarrow \mathbb{R}_+$ an initial state distribution, γ a reward discount factor, and T a fixed horizon. Let $\tau = (s_0, a_0, \dots, s_T, a_T)$ be a trajectory of states and actions and $R(\tau) = \sum_{t=0}^T \gamma^t R(s_t, a_t)$ the trajectory reward. The goal of reinforcement learning methods is to find parameters θ of a policy $\pi_\theta(a|s)$ that maximize the expected discounted reward over trajectories induced by the policy: $\mathbb{E}_{\pi_\theta}[R(\tau)]$ where $s_0 \sim p_0, s_{t+1} \sim P(s_{t+1}|s_t, a_t)$ and $a_t \sim \pi_\theta(a_t|s_t)$.

In our work, the system dynamics are either induced by a simulation engine or real world. As the simulation engine itself is deterministic, a reparameterization trick [35] can be applied to introduce probabilistic dynamics. In particular, we define a distribution of simulation parameters $\xi \sim p_\phi(\xi)$ parameterized by ϕ . The resulting probabilistic system dynamics of the simulation engine are $P_{\xi \sim p_\phi} = P(s_{t+1}|s_t, a_t, \xi)$.

As it was shown in [6, 7, 9], it is possible to design a distribution of simulation parameters $p_\phi(\xi)$, such that a policy trained on $P_{\xi \sim p_\phi}$ would perform well on a real world dynamics distribution. This approach is also known as *domain randomization* and the policy training maximizes the expected reward under the dynamics induced by the distribution of simulation parameters $p_\phi(\xi)$:

$$\max_{\theta} \mathbb{E}_{P_{\xi \sim p_\phi}} [\mathbb{E}_{\pi_\theta}[R(\tau)]] \quad (1)$$

Domain randomization requires a significant expertise and tedious manual fine-tuning to design the simulation parameter distribution $p_\phi(\xi)$. Furthermore, as we show in our experiments, it is often disadvantageous to use overly wide distributions of simulation parameters as they can include scenarios with infeasible solutions that hinder successful policy learning, or lead to exceedingly conservative policies. Instead, in the next section, we present a way to automate the learning of $p_\phi(\xi)$ that makes it possible to shape a suitable randomization without the need to train on very wide distributions.

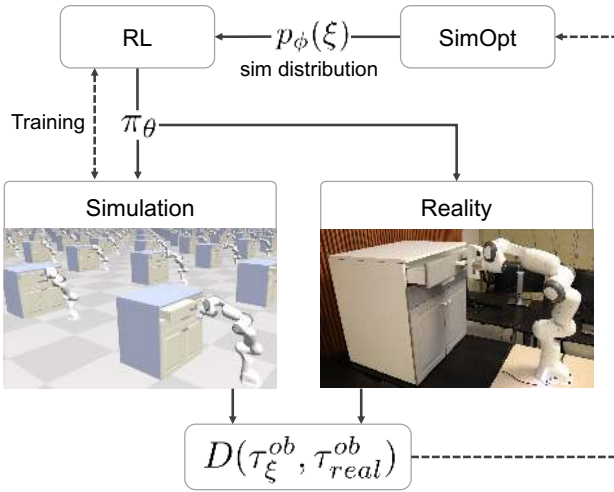


Fig. 3. The pipeline for optimizing the simulation parameter distribution. After training a policy on current distribution, we sample the policy both in the real world and for a range of parameters in simulation. The discrepancy between the simulated and real observations is used to update the simulation parameter distribution in SimOpt.

B. Learning simulation randomization

The goal of our framework is to find a distribution of simulation parameters that brings observations or partial observations induced by the policy trained under this distribution closer to the observations of the real world. Let π_{θ, p_ϕ} be a policy trained under the simulated dynamics distribution $P_{\xi \sim p_\phi}$ as in the objective (1), and let $D(\tau_\xi^{ob}, \tau_{real}^{ob})$ be a measure of discrepancy between real world observation trajectories $\tau_{real}^{ob} = (o_{0,real}, \dots, o_{T,real})$ and simulated observation trajectories $\tau_\xi^{ob} = (o_{0,\xi}, \dots, o_{T,\xi})$ sampled using policy π_{θ, p_ϕ} and the dynamics distribution $P_{\xi \sim p_\phi}$. It should be noted that the inputs of the policy π_{θ, p_ϕ} and observations used to compute $D(\tau_\xi^{ob}, \tau_{real}^{ob})$ are not required to be the same. The goal of optimizing the simulation parameter distribution is to minimize the following objective:

$$\min_{\phi} \mathbb{E}_{P_{\xi \sim p_\phi}} \left[\mathbb{E}_{\pi_{\theta, p_\phi}} [D(\tau_\xi^{ob}, \tau_{real}^{ob})] \right] \quad (2)$$

This optimization would entail training and real robot evaluation of the policy π_{θ, p_ϕ} for each ϕ . This would require a large amount of RL iterations and more critically real robot trials. Hence, we develop an iterative approach to approximate the optimization by training a policy $\pi_{\theta, p_{\phi_i}}$ on the simulation parameter distribution from the previous iteration and using it for both, sampling the real world observations and optimizing the new simulation parameter distribution $p_{\phi_{i+1}}$:

$$\begin{aligned} \min_{\phi_{i+1}} \mathbb{E}_{P_{\xi_{i+1} \sim p_{\phi_{i+1}}}} \left[\mathbb{E}_{\pi_{\theta, p_{\phi_i}}} [D(\tau_{\xi_{i+1}}^{ob}, \tau_{real}^{ob})] \right] \quad (3) \\ \text{s.t. } D_{KL}(p_{\phi_{i+1}} \| p_{\phi_i}) \leq \epsilon, \end{aligned}$$

where we introduce a KL-divergence step ϵ between the old simulation parameter distribution p_{ϕ_i} and the updated distribution $p_{\phi_{i+1}}$ to avoid going out of the trust region of the policy $\pi_{\theta, p_{\phi_i}}$ trained on the old simulation parameter distribution. Fig. 3 shows the general structure of our algorithm that we call *SimOpt*.

C. Implementation

Here we describe particular implementation choices for the components of our framework used in this work. How-

Algorithm 1 SimOpt framework

- 1: $p_{\phi_0} \leftarrow$ Initial simulation parameter distribution
- 2: $\epsilon \leftarrow$ KL-divergence step for updating p_ϕ
- 3: **for** iteration $i \in \{0, \dots, N\}$ **do**
- 4: $\text{env} \leftarrow \text{Simulation}(p_{\phi_i})$
- 5: $\pi_{\theta, p_{\phi_i}} \leftarrow \text{RL}(\text{env})$
- 6: $\tau_{real}^{ob} \sim \text{RealRollout}(\pi_{\theta, p_{\phi_i}})$
- 7: $\xi \sim \text{Sample}(p_{\phi_i})$
- 8: $\tau_\xi^{ob} \sim \text{SimRollout}(\pi_{\theta, p_{\phi_i}}, \xi)$
- 9: $c(\xi) \leftarrow D(\tau_\xi^{ob}, \tau_{real}^{ob})$
- 10: $p_{\phi_{i+1}} \leftarrow \text{UpdateDistribution}(p_{\phi_i}, \xi, c(\xi), \epsilon)$

ever, it should be noted that each of the components is replaceable. Algorithm 1 describes the order of running all the components in our implementation. The RL training is performed on a GPU based simulator using a parallelized version of proximal policy optimization (PPO) [36] on a multi-GPU cluster [37]. We parameterize our simulation parameter distribution as a Gaussian, i.e. $p_\phi(\xi) \sim \mathcal{N}(\mu, \Sigma)$ with $\phi = (\mu, \Sigma)$. We choose weighted ℓ_1 and ℓ_2 norms between simulation and real world observations for our observation discrepancy function D :

$$D(\tau_\xi^{ob}, \tau_{real}^{ob}) = w_{\ell_1} \sum_{i=0}^T |W(o_{i,\xi} - o_{i,real})| + w_{\ell_2} \sum_{i=0}^T \|W(o_{i,\xi} - o_{i,real})\|_2^2, \quad (4)$$

where w_{ℓ_1} and w_{ℓ_2} are the weights of the ℓ_1 and ℓ_2 norms, and W are the importance weights for each observation dimension. We additionally apply a Gaussian filter to the distance computation to account for misalignments of the trajectories.

As we use a non-differentiable simulator we employ a sampling-based gradient-free algorithm based on relative entropy policy search [38] for optimizing the objective (3), which is able to perform updates of p_ϕ with an upper bound on the KL-divergence step. By doing so, the simulator can be treated as a black-box, as in this case p_ϕ can be optimized directly by only using samples $\xi \sim p_\phi$ and the corresponding costs $c(\xi)$ coming from $D(\tau_\xi^{ob}, \tau_{real}^{ob})$. Sampling of simulation parameters and the corresponding policy roll-outs is highly parallelizable, which we use in our experiments to evaluate large amounts of simulation parameter samples.

As noted above, single components of our framework can be exchanged. In case of availability of a differentiable simulator, the objective (3) can be defined as a loss function for optimizing with gradient descent. Furthermore, for cases where ℓ_1 and ℓ_2 norms are not applicable, we can employ other forms of discrepancy functions, e.g. to account for potential domain shifts between observations [16, 39, 40]. Alternatively, real world and simulation data can be additionally used to train $D(\tau_\xi^{ob}, \tau_{real}^{ob})$ to discriminate between the observations by minimizing the prediction loss of classifying observations as simulated or real, similar to the discriminator training in the generative adversarial framework [41, 42, 43]. Finally, a higher-dimensional generative model $p_\phi(\xi)$ can be employed to provide a multi-modal randomization of the simulated environments.

IV. EXPERIMENTS

In our experiments we aim at answering the following questions: (1) How does our method compare to standard domain randomization? (2) How learning a simulation parameter distribution compares to training on a very wide parameter distribution? (3) How many *SimOpt* iterations and real world trials are required for a successful transfer of robotic manipulation policies? (4) Does our method work for different real world tasks and robots?

We start by performing an ablation study in simulation by transferring policies between scenes with different initial state distributions, such as different poses of the cabinet in the drawer opening task. We demonstrate that updating the distribution of simulation parameters leads to a successful policy transfer in contrast to just using an initial distribution of the parameters without any updates as done in standard domain randomization. As we observe, training on very wide parameter distributions is significantly more difficult and prone to fail compared to initializing with a conservative parameter distribution and updating it using *SimOpt* afterwards.

Next, we show that we can successfully transfer policies to real robots, such as ABB Yumi and Franka Panda, for complex articulated tasks such as cabinet drawer opening, and tasks with non-rigid bodies and complex dynamics, such as swing-peg-in-hole task with the peg swinging on a soft rope. The policies can be transferred with a very small amount of real robot trials and leveraging large-scale training on a multi-GPU cluster.

A. Tasks

We evaluate our approach on two robot manipulation tasks: cabinet drawer opening and swing-peg-in-hole.

1) *Swing-peg-in-hole*: The goal of this task is to put a peg attached to a robot hand on a rope into a hole placed at a 45 degrees angle. Manipulating a soft rope leads to a swinging motion of the peg, which makes the dynamics of the task more challenging. The task set up in the simulation and real world using a 7-DoF Yumi robot from ABB is depicted in Fig. 1 on the right. Our observation space consists of 7-DoF arm joint configurations and 3D position of the peg. The *reward function* for the RL training in simulation includes the distance of the peg from the hole, angle alignment with the hole and a binary reward for solving the task.

2) *Drawer opening*: In the drawer opening task, the robot has to open a drawer of a cabinet by grasping and pulling it with its fingers. This task involves an ability to handle contact dynamics when grasping the drawer handle. For this task, we use a 7-DoF Panda arm from Franka Emika. Simulated and real world settings are shown in Fig. 1 on the left. This task is operated on a 10D observation space: 7D robot joint angles and 3D position of the cabinet drawer handle. The *reward function* consists of the distance penalty between the handle and end-effector positions, the angle alignment of the end-effector and the drawer handle, the opening distance of the drawer and an indicator function ensuring that both robot fingers are on the handle.

We would like to emphasize that our method does not require the full state information of the real world, *e.g.* we do not need to estimate the rope diameter, rope compliance *etc.*

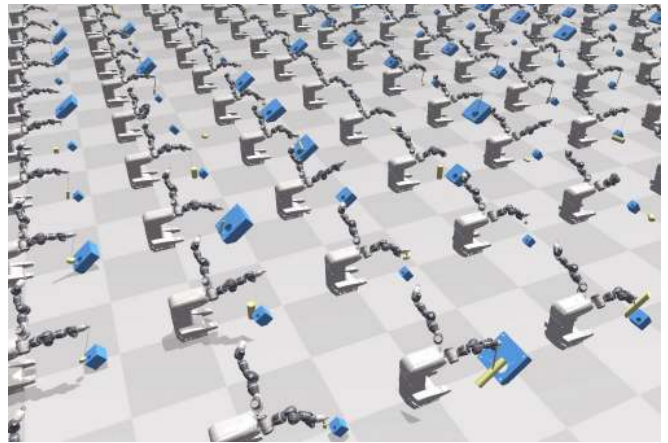


Fig. 4. An example of a wide distribution of simulation parameters in the swing-peg-in-hole task where it is not possible to find a solution for many of the task instances.

to update the simulation parameter distribution in the swing-peg-in-hole task. The output of our policies consists of 7 joint velocity commands and an additional gripper command for the drawer opening task.

B. Simulation engine

We use NVIDIA Flex as a high-fidelity GPU based physics simulator that uses maximal coordinate representation to simulate rigid body dynamics. Flex allows a highly parallel implementation and can simulate multiple instances of the scene on a single GPU. We use the multi-GPU based RL infrastructure developed in [37] to leverage the highly parallel nature of the simulator.

C. Comparison to standard domain randomization

We aim at understanding what effect a wide simulation parameter distribution can have on learning robust policies, and how we can improve the learning performance and the transferability of the policies using our method to adjust simulation randomization. Fig. 4 shows an example of training a policy on a significantly wide distribution of simulation parameters for the swing-peg-in-hole task. In this case, peg size, rope properties and size of the peg box were randomized. As we can observe, a large part of the randomized instances does not have a feasible solution, *i.e.* when the peg is too large for the hole or the rope is too short. Finding a suitably wide parameter distribution would require manual fine-tuning of the randomization parameters.

Moreover, learning performance of standard domain randomization depends strongly on the variance of the parameter distribution. We investigate this in a simulated cabinet drawer opening task with a Franka arm which is placed in front of a cabinet. We randomize the position of the cabinet along the lateral direction (X-coordinate) while keeping all other simulation parameters constant. We train our policies on a 2 layer neural network with fully connected layers of 64 units each with PPO for 200 iterations. As we increase the variance of the cabinet position, we observe that the policies learned tend to be conservative *i.e.* they do end up reaching the handle of the drawer but fail to open it. This is shown in Fig. 5 where we plot the reward as a function of number of iterations used to train the RL policy. We start with a

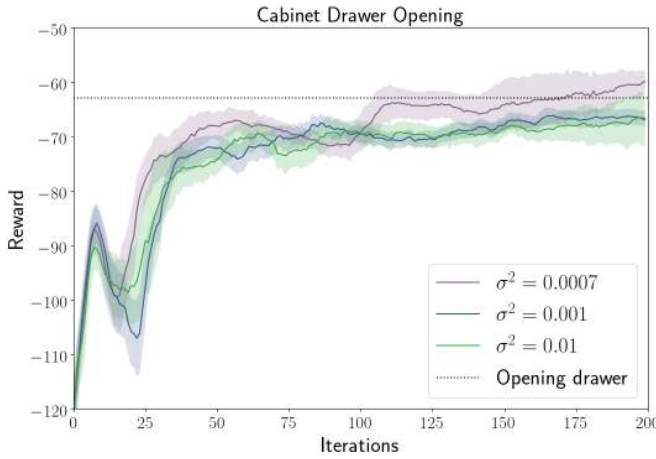


Fig. 5. Performance of the policy training with standard domain randomization for different variances of the distribution of the cabinet position along the X-axis in the drawer opening task.

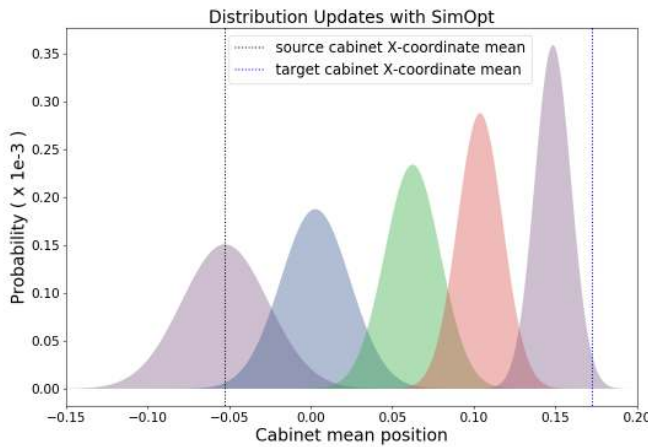


Fig. 6. Initial distribution of the cabinet position in the source environment, located at extreme left, slowly starts to change to the target environment distribution as a function of running 5 iterations of SimOpt.

standard deviation of 2cm ($\sigma^2 = 7e-4$) and increase it to 10cm ($\sigma^2 = 0.01$). As shown in the plot, the policy is sensitive to the choice of this parameter and only manages to open the drawer when the standard deviation is 2cm. We note that the reward difference may not seem that significant but realize that it is dominated by the reaching reward. Increasing variance further, in an attempt to cover a wider operating range, can often lead to simulating unrealistic scenarios and catastrophic breakdown of the physics simulation with various joints of the robot reaching their limits. We also observed that the policy is extremely sensitive to the variance in all three axes of the cabinet position *i.e.* policy only ever converges when the standard deviation is 2cm and fails to learn even reaching the handle otherwise.

In our next set of experiments, we show that our method is able to perform policy transfer from the source to target drawer opening scene where position of the cabinet in the target scene is offset by a distance of 15cm and 22cm. Such large distances would have required the standard deviation of the cabinet position to be at least 10cm for any naïve domain randomization based training which fails to produce a policy that opens the drawer as shown in Fig. 5. The policy is first trained with RL on a conservative initial simulation param-

eter distribution. Afterwards, it is run on the target scene to collect roll-outs. These roll-outs are then used to perform several *SimOpt* iterations to optimize simulation parameters that best explain the current roll-outs. We noticed that the RL training can be sped up by initializing the policy with the weights from the previous *SimOpt* iteration, effectively reducing the number of needed PPO iterations from 200 to 10 after the first *SimOpt* iteration. The whole process is repeated until the learned policy starts to successfully open the drawer in the target scene. We found that it took overall 3 iterations of doing RL and *SimOpt* to learn to open the drawer when the cabinet was offset by 15cm. We further note that the number of iterations increases to 5 as we increase the target cabinet distance to 22cm highlighting that our method is able to operate on a wider range of mismatch between the current scene and the target scene. Fig. 6 shows how the source distribution variance adapts to the target distribution variance for this experiment and Fig. 7 shows that our method starts with a conservative guess of the initial distribution of the parameters and changes it using target scene roll-outs until policy behavior in target and source scenes starts to match.

D. Real robot experiments

In our real robot experiments, *SimOpt* is used to learn simulation parameter distribution of the manipulated objects and the robot. We run our experiments on 7-DoF Franka Panda and ABB Yumi robots. The RL training and *SimOpt* simulation parameter sampling is performed using a cluster of 64 GPUs for running the simulator with 150 simulated agents per GPU. In the real world, we use object tracking with DART [44] to continuously track the 3D positions of the peg in the swing-peg-in-hole task and the handle of the cabinet drawer in the drawer opening task, as well as initialize positions of the peg box and the cabinet in simulation. DART operates on depth images and requires 3D articulated models of the objects. We learn multi-variate Gaussian distributions of the simulation parameters parameterized by a mean and a full covariance matrix, and perform several updates of the simulation parameter distribution per *SimOpt* iteration using the same real world roll-outs to minimize the number of real world trials.

1) *Swing-peg-in-hole*: Fig. 8 (left) demonstrates the behavior of real robot execution of the policy trained in simulation over 3 iterations of *SimOpt*. At each iteration, we perform 100 iterations of RL in approximately 7 minutes and 3 roll-outs on the real robot using the currently trained policy to collect real world observations. Then, we run 3 update steps of the simulation parameter distribution with 9600 simulation samples per update. In the beginning, the robot misses the hole due to the discrepancy of the simulation parameters and the real world. After a single *SimOpt* iteration, the robot is able to get much closer to the hole, however not being able to insert the peg as it requires a slight angle to go into the hole, which is non-trivial to achieve using a soft rope. Finally, after two *SimOpt* iterations, the policy trained on a resulting simulation parameter distribution is able to swing the peg into the hole in 90% of the times when evaluated on 20 trials.

We observe that the most significant changes of the simulation parameter distribution occur in the physical parameters

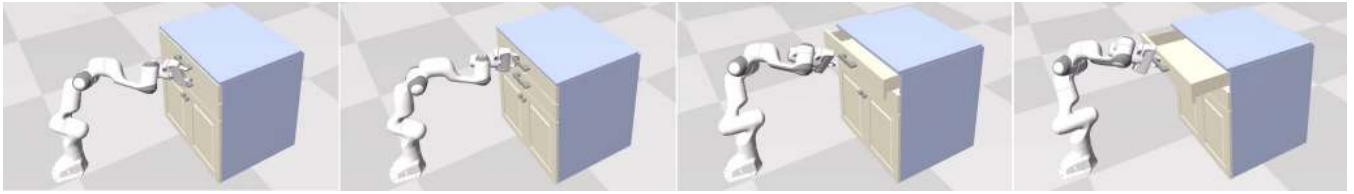


Fig. 7. Policy performance in the target drawer opening environment trained on randomized simulation parameters at different iterations of SimOpt. As the source environment distribution gets adjusted, the policy transfer improves until the robot can successfully solve the task in the fourth SimOpt iteration.

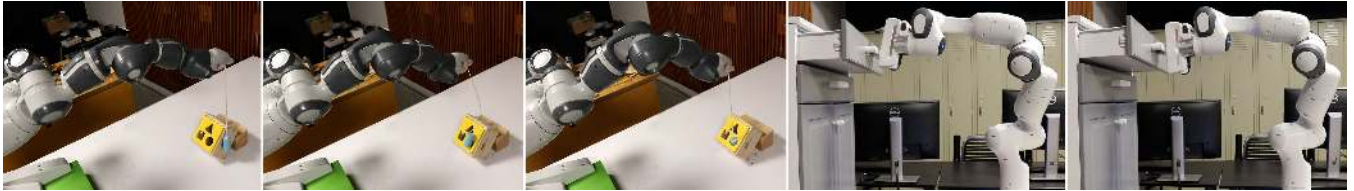


Fig. 8. Running policies trained in simulation at different iterations of SimOpt for real world swing-peg-in-hole and drawer opening tasks. *Left*: SimOpt adjusts physical parameter distribution of the soft rope, peg and the robot, which results in a successful execution of the task on a real robot after two SimOpt iterations. *Right*: SimOpt adjusts physical parameter distribution of the robot and the drawer. Before updating the parameters, the robot pushes too much on the drawer handle with one of its fingers, which leads to opening the gripper. After one SimOpt iteration, the robot can better control its gripper orientation, which leads to an accurate task execution.

of the rope that influence its dynamical behavior and the robot parameters that influence the policy behavior, such as scaling of the policy actions. More details on the initial and updated Gaussian distribution parameters can be found in Appendix B of [45]. Fig. 9 shows the development of the covariance matrix over the iterations. We can observe some correlation in the top left block of the matrix, which corresponds to the robot joint compliance and damping values. This reflects the fact that these values have somewhat opposite effect on the robot behavior, i.e. if we overshoot in the compliance we can compensate with increased damping.

2) *Drawer opening*: For drawer opening, we learn a Gaussian distribution of the robot and cabinet simulation parameters. More details on the learned distribution and its initialization are provided in Appendix B of [45]. Fig. 8 (right) shows the drawer opening behavior before and after performing a *SimOpt* update. During each *SimOpt* iteration, we run 200 iterations of RL for approximately 22 minutes, perform 3 real robot roll-outs and run 20 update steps of the simulation distribution using 9600 samples per update step. Before updating the parameter distribution, the robot is able to reach the handle and start opening the drawer. However, it cannot exactly replicate the learned behavior from simulation and does not keep the gripper orthogonal to the drawer, which results in pushing too much on the handle from the bottom with one of the robot fingers. As the finger gripping force is limited, the fingers begin to open due to a larger pushing force. After adjusting the simulation parameter distribution that includes robot and drawer properties, the robot is able to better control its gripper orientation and by evaluating on 20 trials can open the drawer at all times keeping the gripper orthogonal to the handle.

V. CONCLUSIONS

Closing the simulation to reality transfer loop is an important component for a robust transfer of robotic policies. In this work, we demonstrated that adapting simulation randomization using real world data can help in learning simulation parameter distributions that are particularly suited

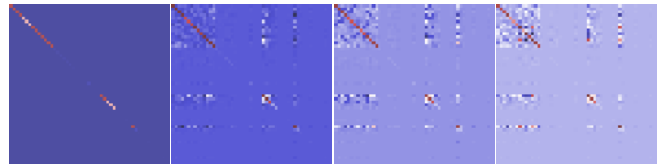


Fig. 9. Covariance matrix heat maps over 3 SimOpt updates of the swing-peg-in-hole task beginning with the initial covariance matrix.

for a successful policy transfer without the need for exact replication of the real world environment. In contrast to trying to learn policies using very wide distributions of simulation parameters, which can simulate infeasible scenarios, we are able to start with distributions that can be efficiently learned with reinforcement learning, and modify them for a better transfer to the real world scenario. Our framework does not require full state of the real environment and reward functions are only needed in simulation. We showed that updating simulation distributions is possible using partial observations of the real world while the full state still can be used for the reward computation in simulation. We evaluated our approach on two real world robotic tasks and showed that policies can be transferred with only a few iterations of simulation updates using a small number of real robot trials.

In this work, we applied our method to learning uni-modal simulation parameter distributions. We plan to extend our framework to multi-modal distributions and more complex generative simulation models in future work. Furthermore, we plan to incorporate higher-dimensional sensor modalities, such as vision and touch, for both policy observations and factors of simulation randomization.

ACKNOWLEDGEMENTS

We would like to thank Alexander Lambert, Balakumar Sundaralingam and Giovanni Suto for their help with the robot experiments, and David Ha, James Davidson, Lerrel Pinto and Fabio Ramos for their helpful feedback on the draft of the paper. We would also like to thank the GPU cluster and infrastructure team at NVIDIA for their help all the way through this project.

REFERENCES

- [1] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *I. J. Robotics Res.*, 37(4-5):421–436, 2018.
- [2] L. Pinto and A. Gupta. Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. In *ICRA*, 2016.
- [3] A. Yahya, A. Li, M. Kalakrishnan, Y. Chebotar, and S. Levine. Collective robot reinforcement learning with distributed asynchronous guided policy search. In *IROS*, 2017.
- [4] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine. Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *CoRR*, abs/1806.10293, 2018.
- [5] N. Jakobi, P. Husbands, and I. Harvey. Noise and the reality gap: The use of simulation in evolutionary robotics. In *European Conference on Artificial Life*. Springer, 1995.
- [6] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IROS*, 2017.
- [7] F. Sadeghi and S. Levine. Cad2rl: Real single-image flight without a single real image. *RSS*, 2017.
- [8] S. James, A. J. Davison, and E. Johns. Transferring end-to-end visuomotor control from simulation to real world for a multi-stage task. *CoRR*, abs/1707.02267, 2017.
- [9] M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *CoRR*, abs/1808.00177, 2018.
- [10] L. Ljung. *System identification – theory for the user*. Prentice Hall, 1999.
- [11] F. Giri and E.-W. Bai. *Block-oriented Nonlinear System Identification*. London: Springer-Verlag London, 2010.
- [12] M. P. Deisenroth, G. Neumann, and J. Peters. A survey on policy search for robotics. *Foundations and Trends in Robotics*, pages 388–403, 2013.
- [13] M. P. Deisenroth and C. E. Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *ICML*, 2011.
- [14] M. P. Deisenroth, C. E. Rasmussen, and D. Fox. Learning to control a low-cost manipulator using data-efficient reinforcement learning. In *RSS*, 2011.
- [15] S. Koley and E. Todorov. Physically consistent state estimation and system identification for contacts. In *Humanoids*, 2015.
- [16] E. Tzeng, C. Devin, J. Hoffman, C. Finn, X. Peng, S. Levine, K. Saenko, and T. Darrell. Towards adapting deep visuomotor representations from simulated to real environments. *CoRR*, abs/1511.07111, 2015.
- [17] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kelcey, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige, S. Levine, and V. Vanhoucke. Using simulation and domain adaptation to improve efficiency of deep robotic grasping. *CoRR*, abs/1709.07857, 2017.
- [18] A. A. Rusu, M. Vecerik, T. Rothl, N. Heess, R. Pascanu, and R. Hadsell. Sim-to-real robot learning from pixels with progressive nets. In *CoRL*, 2017.
- [19] P. F. Christiano, Z. Shah, I. Mordatch, J. Schneider, T. Blackwell, J. Tobin, P. Abbeel, and W. Zaremba. Transfer from simulation to real world through learning deep inverse dynamics model. *CoRR*, abs/1610.03518, 2016.
- [20] S. Koos, J.-B. Mouret, and S. Doncieux. Crossing the reality gap in evolutionary robotics by promoting transferable controllers. In *GECCO*. ACM, 2010.
- [21] I. Mordatch, K. Lowrey, and E. Todorov. Ensemble-cio: Full-body dynamic motion planning that transfers to physical humanoids. In *IROS*, 2015.
- [22] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *ICRA*, 2018.
- [23] W. Yu, J. Tan, C. K. Liu, and G. Turk. Preparing for the unknown: Learning a universal policy with online system identification. In *RSS*, 2017.
- [24] F. Muratore, F. Treede, M. Gienger, and J. Peters. Domain randomization for simulation-based policy optimization with transferability assessment. In *CoRL*, 2018.
- [25] M. Wulfmeier, I. Posner, and P. Abbeel. Mutual alignment transfer learning. *CoRR*, abs/1707.07907, 2017.
- [26] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *RSS*, 2018.
- [27] K. Lowrey, S. Koley, J. Dao, A. Rajeswaran, and E. Todorov. Reinforcement learning for non-prehensile manipulation: Transfer from simulation to physical system. In *SIMPACT*, 2018.
- [28] R. Antonova, S. Cruciani, C. Smith, and D. Kragic. Reinforcement learning for pivoting task. *CoRR*, abs/1703.00472, 2017.
- [29] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel. Asymmetric actor critic for image-based robot learning. *CoRR*, abs/1710.06542, 2017.
- [30] J. Tan, Z. Xie, B. Boots, and C. K. Liu. Simulation-based design of dynamic controllers for humanoid balancing. In *IROS*, 2016.
- [31] S. Zhu, A. Kimmel, K. E. Bekris, and A. Boularias. Fast model identification via physics engines for data-efficient policy search. In *IJCAI*. ijcai.org, 2018.
- [32] A. Farchy, S. Barrett, P. MacAlpine, and P. Stone. Humanoid robots learning to walk faster: From the real world to simulation and back. In *AAMAS*, 2013.
- [33] J. Hanna and P. Stone. Grounded action transformation for robot learning in simulation. In *AAAI*, 2017.
- [34] A. Rajeswaran, S. Ghotra, S. Levine, and B. Ravindran. Epopt: Learning robust neural network policies using model ensembles. *CoRR*, abs/1610.01283, 2016.
- [35] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *CoRR*, abs/1312.6114, 2013.
- [36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.
- [37] J. Liang, V. Makovychuk, A. Handa, N. Chentanez, M. Macklin, and D. Fox. Gpu-accelerated robotic simulation for distributed reinforcement learning. *CoRL*, 2018.
- [38] J. Peters, K. Mlling, and Y. Altun. Relative entropy policy search. In *AAAI*, 2010.
- [39] E. Tzeng, J. Hoffman, T. Darrell, and K. Saenko. Simultaneous deep transfer across domains and tasks. In *ICCV*, 2015.
- [40] P. Sermanet, C. Lynch, Y. Chebotar, J. Hsu, E. Jang, S. Schaal, and S. Levine. Time-contrastive networks: Self-supervised learning from video. In *ICRA*, 2018.
- [41] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. C. Courville, and Y. Bengio. Generative adversarial nets. In *NIPS*, 2014.
- [42] J. Ho and S. Ermon. Generative adversarial imitation learning. In *NIPS*, 2016.
- [43] K. Hausman, Y. Chebotar, S. Schaal, G. S. Sukhatme, and J. J. Lim. Multi-modal imitation learning from unstructured demonstrations using generative adversarial nets. In *NIPS*, 2017.
- [44] T. Schmidt, R. A. Newcombe, and D. Fox. Dart: Dense articulated real-time tracking. In *RSS*, 2014.
- [45] Y. Chebotar, A. Handa, V. Makovychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox. Closing the sim-to-real loop: Adapting simulation randomization with real world experience. *CoRR*, abs/1810.05687, 2018.